sandbox-api

System Design Document

Architecture • Data Flow • SOLID Principles • v1.0.0

Node.js ESM	Express REST	Zoho CRM OAuth2 IN	RDP Disk JSON+Files	SOLID Architecture

Table of Contents

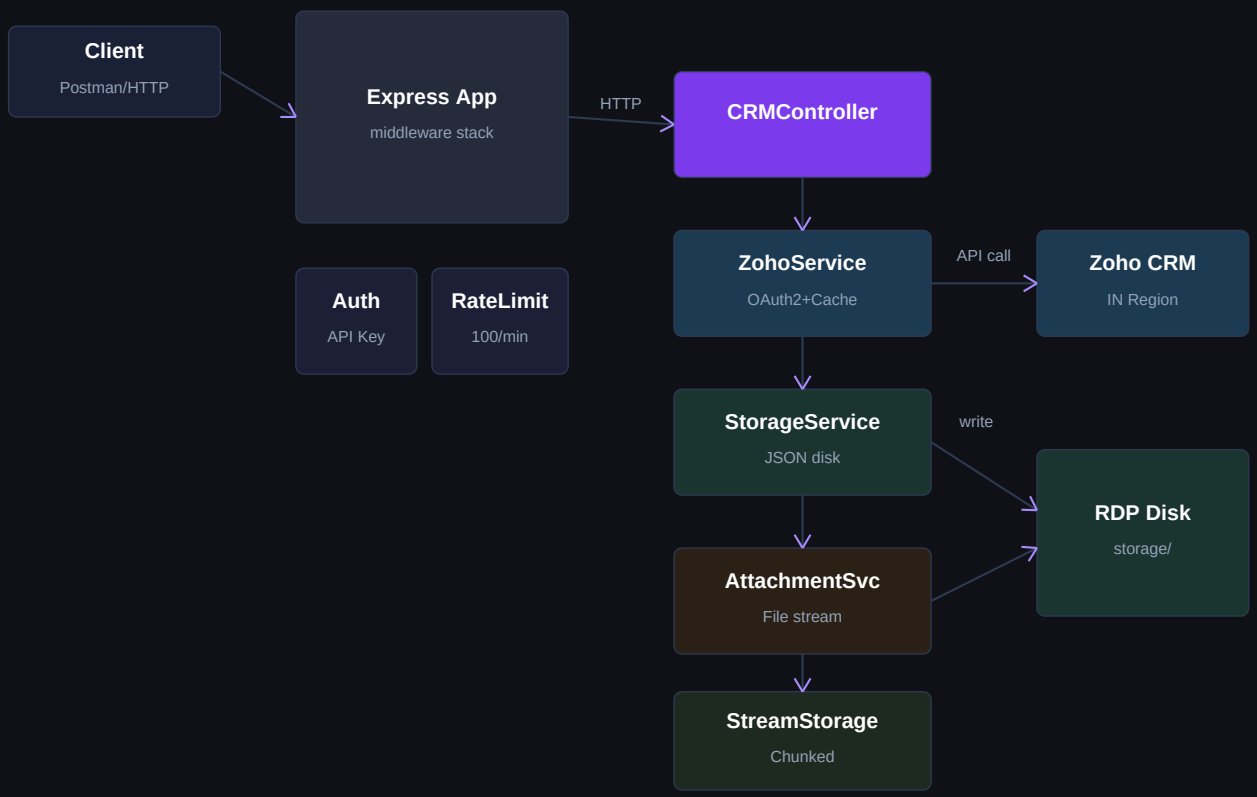
1. System Overview 3
2. Architecture Diagram 3
3. Layer Breakdown 4
4. Request Lifecycle 5
5. Sync Data Pipeline 6
6. Attachment Download Flow 7
7. Storage Layout on Disk 8
8. Memory Management 9
9. Security Model 10
10. SOLID Principles Applied 11
11. Error Handling 12
12. Logging Architecture 12

1. System Overview

sandbox-api is a production-grade Node.js REST API server deployed on a Windows RDP server. It integrates with Zoho CRM (India region) via OAuth2 to fetch leads, contacts, accounts, and deals. Fetched records are stored as JSON files on the RDP disk. Attachments from CRM records are streamed directly to disk without loading into memory. Built with SOLID principles, full dependency injection, ESM modules, and daily-rotating Winston logs.

Goal	Implementation
Zoho CRM Integration	OAuth2 token auto-refresh, paginated fetch, 5-min response cache
RDP Disk Storage	JSON persistence, chunked writes, 30-day snapshot auto-pruning
Attachment Pipeline	Stream files from Zoho to RDP disk — never buffered in memory
High Availability	Sync queue prevents duplicate runs, rate limiting prevents overload
Maintainability	SOLID architecture, interface DI, single composition root in app.js

2. Architecture Diagram



All concrete service implementations are instantiated once in `app.js` (composition root) and injected into controllers via constructor. No controller imports a concrete service directly.

3. Layer Breakdown

Transport	`server.js + app.js`	Express HTTP server, graceful shutdown, Helmet, CORS, compression, Morgan, UUID req IDs, Swagger UI at /api-docs.
Auth/Limit	`middleware/`	API key validation on /api/v1/*. Standard: 100/min. Strict: 10/min on sync + download.
Routing	`routes/crm.js`	Route factory pattern. Routes accept controller instances via argument — never import them directly.
Controller	`controllers/`	Validates HTTP input via Joi, delegates to services, shapes responses. Zero business logic. Zero concrete imports.
Services	`services/`	ZohoService (CRM+OAuth2), StorageService (JSON disk), AttachmentService (file stream), StreamStorageService (chunks), SyncFreshnessPolicy.
Interfaces	`interfaces/`	ICRMService, IStorageService, IAttachmentService — abstract base classes enabling Liskov + Dependency Inversion.
Infrastructure	`store/ + utils/`	CacheStore (TTL cache), SyncQueue (per-module lock), logger (Winston daily-rotate), validator (Joi schemas).

4. Request Lifecycle

Every /api/v1/* request passes through this middleware chain before reaching the controller. /health and /api-docs are public and skip auth + rate limit.

1	Incoming Request	Client sends HTTP request with X-API-Key header
2	helmet()	Sets CSP, HSTS, X-Frame-Options, and 8 other security headers
3	cors()	Validates origin. Prod: whitelist. Dev: allow all.
4	compression()	Gzip compresses response body automatically
5	express.json()	Parses request body. Max 10MB.
6	Request ID	Assigns UUID to req.id for log correlation
7	Morgan Logger	Logs HTTP access to rotating http log file
8	apiLimiter	429 if over 100 req/min per IP
9	apiKeyAuth	401 if X-API-Key missing or invalid
10	Controller	Joi validation → service call → JSON response
11	errorHandler	Catches all errors, returns structured JSON

5. Sync Data Pipeline

Triggered by POST /api/v1/crm/sync. Fetches all records via auto-pagination then writes to disk in two formats concurrently.

1	**Validate**	Joi validates { module, overwrite } in request body.
2	**Queue check**	SyncQueue.isRunning(module) — returns 409 if already running.
3	**Freshness**	SyncFreshnessPolicy.isFresh() — skips sync if data < 30 min old and overwrite=false.
4	**Lock**	SyncQueue.start(module) — marks module as syncing.
5	**Fetch Zoho**	ZohoService.fetchAllRecords() auto-paginates at 200/page until more_records=false. Token auto-refreshed on expiry.
6	**Write (parallel)**	Promise.all([StorageService.save() → latest.json + snapshot, StreamStorageService.saveChunked() → meta.json + chunk_NNN.json files])
7	**Unlock**	SyncQueue.finish(module) in finally block — always runs even on error.
8	**Respond**	{ success, module, count, chunkCount, latestPath, syncedAt }

6. Attachment Download Flow

Attachments are not in standard CRM record responses. They need a dedicated fetch and download pipeline. Files are piped directly from Zoho to RDP disk.

1	**List Attachments**	GET /crm/v2/{module}/{recordId}/Attachments — metadata array: id, File_Name, Size. 204 = no attachments (not an error).
2	**Per Attachment**	downloadAttachment(module, recordId, attachmentId, fileName) called for each item.
3	**Ensure Dir**	fse.ensureDir(storage/attachments/{module}/{recordId}/) — creates path if missing.
4	**Stream to disk**	axios responseType:"stream" + pipe() — bytes flow from Zoho network directly to disk write stream. Never fully in memory.
5	**Verify**	fse.stat() confirms file size. Returns { attachmentId, fileName, filePath, size }.

7. Storage Layout on Disk

All data stored under two root paths set in .env.

```
storage/
├── crm-data/
│   ├── Leads/
│   │   ├── latest.json
│   │   ├── meta.json
│   │   ├── chunk_001.json
│   │   ├── chunk_002.json ...
│   │   └── snapshots/
│   │       ├── 2026-03-10T10-09-55.json
│   │       └── 2026-03-09T08-00-00.json
└── attachments/
    └── Leads/
        └── 1113843000000696206/
            ├── document.pdf
            └── contract.pdf
```

File	Purpose
`latest.json`	Overwritten on every sync. Full records + fetchedAt + count.
`meta.json`	Lightweight index. totalCount, chunkCount, chunkSize, fetchedAt. Always safe to load.
`chunk_NNN.json`	500 records per file. Only one chunk loaded per paged request.
`snapshots/*.json`	Immutable point-in-time copy. Auto-pruned after 30 days.
`attachments/**`	Files streamed from Zoho. Named as-is from CRM. Grouped by recordId.

8. Memory Management Strategy

Four mechanisms ensure the server never spikes memory regardless of dataset size.

Chunked Writes	StreamStorageService.saveChunked() slices records array into 500-record chunks. Only one chunk is in the fse.writeJson() call at a time. OS can page the rest.
Chunked Reads	readPage() loads exactly one chunk file per API request. For 50,000 records across 100 chunks, page 1 only reads chunk_001.json — never the full 50k.
Streaming Attachments	axios responseType:"stream" + pipe(). File bytes travel from Zoho network socket directly to a disk write stream. Only a small TCP buffer exists in heap at any moment.
Response Cache	CacheStore caches ZohoService.fetchRecords() per unique query for 5 minutes. Repeated identical requests return the cached object — no new allocation.

9. Security Model

API Key Auth	X-API-Key header required on all /api/v1/*. Stored in .env only, never committed.
Helmet Headers	CSP, X-Frame-Options, X-Content-Type-Options, HSTS, and 8 more auto-set.
Rate Limiting	express-rate-limit: 100/min standard, 10/min on sync+download. Returns 429.
Zoho Token Security	Refresh token in .env only. Access token in CacheStore memory (never on disk). Auto-cleared on 401.
CORS Policy	Production: empty whitelist. Development: allow all. Controlled by NODE_ENV.
Error Leakage	4xx errors expose message. 5xx returns generic text only. Stack traces in dev logs only.

10. SOLID Principles Applied

Principle	How It Is Applied
S — Single Responsibility	Each class owns one concern. ZohoService = Zoho API only. StorageService = disk I/O only. SyncFreshnessPolicy = staleness rule only.
O — Open / Closed	SyncFreshnessPolicy open for extension (subclass to change strategy), closed for modification. Controllers never change when rules change.
L — Liskov Substitution	ZohoService extends ICRMService. StorageService extends IStorageService. AttachmentService extends IAttachmentService. Each is a valid substitute.
I — Interface Segregation	Three focused interfaces: ICRMService, IStorageService, IAttachmentService. No class implements methods it does not need.
D — Dependency Inversion	Controllers receive interfaces via constructor. app.js is the ONLY file where concrete classes are instantiated (composition root).

Swapping ZohoService for a different CRM provider requires changing exactly one import line in `app.js`. Zero other files change.

11. Error Handling Strategy

All errors are routed through the global errorHandler middleware. Controllers call next(err). Unhandled rejections in server.js log and exit with code 1.

Error Type	Where Thrown	HTTP	Exposed?
Validation Error	`Controller (Joi)`	`400`	Yes — full message
Not Found	`Controller`	`404`	Yes — resource name
Unauthorized	`rateLimiter.js`	`401`	Yes — generic message
Sync Conflict	`Controller`	`409`	Yes — module + startedAt
Rate Limit	`rateLimiter.js`	`429`	Yes — generic message
Zoho API Error	`ZohoService`	`502`	No — generic 500
Disk Write Error	`StorageService`	`500`	No — generic 500

12. Logging Architecture

Winston with winston-daily-rotate-file. Three separate rotating log files. All logs are structured JSON for easy parsing.

File Pattern	Level	Retention	Content

`combined-YYYY-MM-DD.log`	`info+`	30 days	All info, warn, error structured logs
`error-YYYY-MM-DD.log`	`error`	30 days	Error-only log for quick diagnosis
`http-YYYY-MM-DD.log`	`http`	14 days	Morgan HTTP access logs: method, URL, status, ms